

MULTIMEDIA ENGINEERING

Implementation of JPEG Image Compression using the Discrete Cosine Transform (DCT)



Compiled By:

Zayyanu Awwal

L0123147

INFORMATICS STUDY PROGRAM

FACULTY OF INFORMATION TECHNOLOGY AND DATA SCIENCE

UNIVERSITAS SEBELAS MARET SURAKARTA

2026

1. Introduction

Digital images are widely used in multimedia applications, including communication, education, entertainment, and information systems. As image resolution increases, the amount of storage space and transmission bandwidth required also becomes larger. Therefore, image compression is an important technique in multimedia engineering because it reduces image size while maintaining acceptable visual quality.

According to the Multimedia Engineering course material, JPEG image compression consists of three coding systems: Baseline Lossy Coding, Extended Coding, and Lossless Coding. This project implements the Baseline Lossy JPEG method because it is the standard JPEG compression technique and directly applies the Discrete Cosine Transform (DCT). Unlike the Lossless JPEG method, which focuses on perfect reconstruction, or the Extended JPEG mode, which supports additional features such as progressive reconstruction, the baseline method provides a simpler implementation while achieving a good balance between image quality and compression efficiency.

The Discrete Cosine Transform converts image data from the spatial domain into the frequency domain, allowing less significant high-frequency information to be reduced through the quantization process. In this project, the mathematical equations of the two-dimensional Discrete Cosine Transform (2D-DCT) and the Inverse Discrete Cosine Transform (IDCT) are implemented manually in Python without using built-in DCT functions. Pillow, NumPy, and Matplotlib are used only for image loading, matrix operations, and visualization. The implementation is evaluated using Mean Squared Error (MSE), Peak Signal-to-Noise Ratio (PSNR), and the estimated compression ratio.

2. Objectives

To Implement, understand and evaluate DCT JPEG compression mathematically from scratch

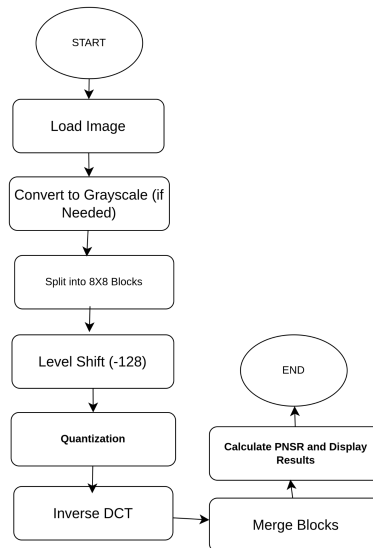
3. Methodology

Development Environment

Software	Purpose
Python3	Programming Language
VS Code	Development Environment
Pillow	Reading Images

Numpy	Matrix Operations
Matplotlib	Result Visualization

JPEG Compression Workflow



JPEG Image Compression Workflow

The implementation follows the standard baseline JPEG compression procedure shown in the figure above. After the user selects an input image, the program verifies that the image is in grayscale format before dividing it into non-overlapping 8 X 8 pixel blocks. Each block undergoes a level shift by subtracting 128 from every pixel so that the pixel values are centered around zero. The shifted image blocks are transformed into the frequency domain using the manually implemented two dimensional discrete cosine transform (2D - DCT). The resulting frequency coefficients are then quantized using the standard JPEG luminance quantization matrix to remove less significant high-frequency information. After quantization, inverse quantization and the inverse discrete cosine transform are applied to reconstruct the image. Finally, the reconstructed blocks are then merged to form the output image, then the compression quality is evaluated using the MSE, PNSR, the estimated compression ratio

DCT Mathematical Model

Manual Implementation of two-dimensional discrete cosine transform (2D - DCT). The program directly evaluates the mathematical equation presented in the course material. Since JPEG compression always processes fixed 8 X 8 image blocks, the implementation substitutes $m=n=8$ into the general DCT equation.

$$I(y, x) = \frac{2}{\sqrt{nm}} \sum_{v=0}^{m-1} \sum_{u=0}^{n-1} F(v, u) C(u) C(v) \cos\left(\frac{(2x+1)u\pi}{2n}\right) * \cos\left(\frac{(2y+1)v\pi}{2m}\right)$$

2D Discrete Cosine Transform

The DCT coefficient above represents the spatial information of an image block in the frequency domain. Most of the important visual information concentrated in the low-frequency coefficients located in the upper-left corner of the DCT matrix. This property enables efficient compression by reducing less significant high-frequency coefficients during the quantization stage.

$$T(x, y) = \text{Round}\left\{\frac{T(x, y)}{QM(x, y)}\right\}$$

JPEG Quantization

After computing the DCT coefficients, each coefficient is now divided by the corresponding value in the JPEG quantization matrix and rounded to the nearest integer. Larger quantization values remove more high-frequency information and increase the compression ratio, whereas smaller values preserve more image details and produce higher reconstruction quality

Program Implementation

The complete implementation source code is available on the attached github repo: https://github.com/Zeryyern/DCT_JPEG_Compression.git

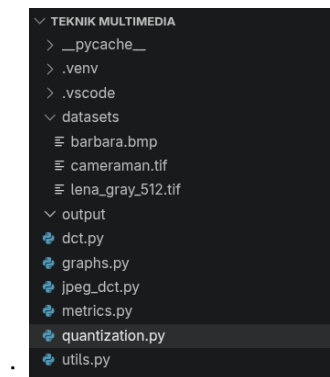


Figure 1. Project Folder Structure

Project Structure JPEG Compression

The implementation is organized into several python modules to improve readability and code maintainability. Each module is responsible for a specific stage of the JPEG compression process, including DCT Computation, quantization, performance evaluation, visualization and a simple CLI user interaction.

[dct.py](#)

The dct module implements the FDCT and IDCT, which form the core of the JPEG compression algorithm. The transform is implemented directly from the 2D DCT equation in the material using nested loops and cosine calculations instead of using built in DCT functions. The FDCT converts each 8X8 image block from the spatial domain into the frequency domain, while IDCT reconstructs the image block after inverse quantization.

[quantization.py](#)

This module performs JPEG quantization and inverse quantization processes. During quantization, each DCT coefficient is divided by the corresponding element in the standard JPEG luminance quantization matrix and rounded to the nearest integer. This stage reduces high - frequency information and achieves image compression.

[metrics.py](#)

It evaluates graphical outputs that assists in easier analyzing the compression results. It displays the original image, reconstructed image, difference image, and histogram for comparison.

4. Results and Discussion

The implemented JPEG compression algorithm was evaluated using the Lena grayscale benchmark image. The experiment was conducted by compressing the image using five different JPEG quality factors (20, 40, 60, 75, and 90). For each quality factor, the Mean Squared Error (MSE), Peak Signal-to-Noise Ratio (PSNR), and estimated compression ratio were recorded to evaluate the relationship between image quality and compression efficiency.

Table 4.1 Compression performance at different JPEG quality factors

Quality	MSE	PSNR (db)	Compression Ratio
20	32.8415	32.97	16.57 : 1
40	19.8679	35.15	10.67 : 1
60	14.6241	36.48	8.0 :1
75	10.6287	37.87	5.96 :1
90	5.2963	40.89	3.38 :1

Table 4.1 shows that the JPEG quality factor has a direct influence on both compression performance and reconstructed image quality. As the quality factor increases, the MSE decreases while the PSNR increases, indicating that the reconstructed image becomes closer to the original image. However, the compression ratio also decreases because fewer DCT coefficients are discarded during quantization. This behavior is consistent with the fundamental principle of JPEG compression, where higher image quality is obtained at the expense of lower compression efficiency.



Figure 2: Comparison between original and reconstructed image

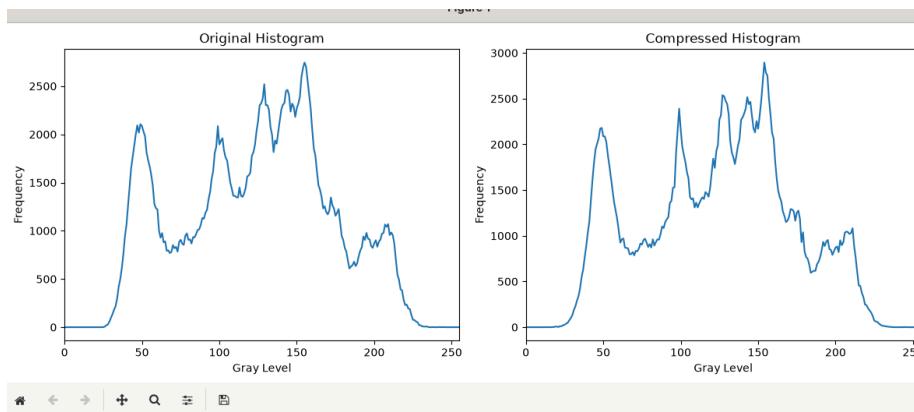


Figure 3 Histogram comparison between the original image and reconstructed image

CONCLUSION

This project successfully implements the standard Baseline JPEG compression algorithm by manually applying the mathematical equations of the two-dimensional Discrete Cosine Transform (2D-DCT), quantization, inverse quantization, and the Inverse Discrete Cosine Transform (IDCT). The implementation demonstrates how image information can be transformed from the spatial domain into the frequency domain and compressed by reducing less significant frequency components.

Experimental results obtained using the Lena, Barbara, and Cameraman benchmark images show that JPEG compression effectively reduces image information while preserving acceptable visual quality. The calculated MSE, PSNR, and estimated compression ratio confirm that the quality factor directly affects the balance between image quality and compression performance. Overall, this assignment provides a practical understanding of the mathematical concepts behind JPEG compression and demonstrates how the DCT can be implemented without relying on built-in transform functions.

Reference

- [1] Heri Prasetyo, "Image JPEG Compression Using Discrete Cosine Transform," Multimedia Engineering Course Notes, Informatics Study Program, Faculty of Information Technology and Data Science, Universitas Sebelas Maret, 2026.
- [2] R. C. Gonzalez and R. E. Woods, Digital Image Processing, 4th ed. Pearson, 2018.
- [3] W. B. Pennebaker and J. L. Mitchell, JPEG Still Image Data Compression Standard. New York, NY, USA: Springer, 1993.